

小作文

自我介绍

面试官你好，我叫毛智明 我来自广东梅州 我毕业于广东工业大学华立学院计算机专业 从实习开始一直在贵州数算互联科技有限公司从事大数据开发的工作 到现在有三年多的工作经验。

大家好，我有 3 年左右的数据仓库开发经验。嗯.....主要做的是 Hadoop 生态和阿里云大数据平台这块。

平时工作基本都是离线数仓开发。从数据接入到最后报表展示，这一整套流程我都参与过。数据同步主要用 Sqoop 和 DataX，数据处理的话，平时还是 Hive 和 MaxCompute 用得比较多，负责数仓建模、ETL 开发和指标计算，Hive SQL 写得比较多，也会写一些 Shell 脚本。调度这块用过 DolphinScheduler、Azkaban，还有 DataWorks，报表的话做过 FineReport 和 Quick BI。

整体来说，我从 ODS 到 ADS 这几层都做过，也能独立完成需求分析、模型设计、开发上线这些工作。

最近这几年主要还是做保险行业，也接触过一些银行业务。像华安财险收展人员经营考核项目，还有都邦财险客户画像和精准营销项目，我都参与过。

在这些项目里面，我主要负责数据抽取、清洗、分层建模、指标开发，还有调度配置和报表展示，基本算是从开发到上线全程参与。技术栈也覆盖了 Oracle、Hive、Spark、MaxCompute 这些。

我觉得这几年的收获，更多还是对整个离线数仓开发流程有了比较完整的理解。接到需求以后，我一般会先分析业务，再去设计模型和开发，而不是直接写 SQL。我也希望后面能继续往大数据和数据仓库方向深入发展，不断提升自己的能力。

第一个项目

最近项目做的是华安财险的有关收展人员业务业绩有关的项目，本项目主要就是以统计收展员在日常工作当中的展业和销售方面的业务统计，像统计产品、人员拜访、参会、出单。

我们分有产品经理、测试工程师、运维工程师、我属于数据开发团队，我们团队大概有 8 个人，1 名小组组长（分配任务、对接甲方人等），2 人做需求分析，4 人做数据开发，1 人做运维

核心是开发是开发保险代理人的展业数据分析、新人、老人奖励机制等需求，**技术栈整体是 **Hadoop HDFS + Hive + DataX+ Shell 脚本调度 + FineReport**，纯离线架构，没有用到云服务。**

额为什么这样呢？这样选型主要有两个原因：一是华安是金融企业，数据安全合规要求高，核心数据不出本地机房；二是保险考核数据的时效需求是 T+1 就足够了，不需要实时流处理，离线批量完全可以满足每天早上 8 点前出数的要求。

整体分层是标准的四层：ODS 贴源层、DWD 明细宽表层、DWS 汇总层、ADS 应用层。ETL 用 Hive SQL 来写，调度 **DolphinScheduler**，每天早上凌晨 2 点开始跑，6 点前跑完，8 点前 FineReport 报表刷新完成。

项目上线时间：两周更新 日常维护调度

工作流程

"我们整个项目，其实都是按照需求一步一步往下走的。

首先会开项目启动会，甲方业务部门会先介绍他们这次项目要实现什么目标，比如要做哪些指标、哪些报表，需要分析哪些业务。

启动会结束以后，我们会再和甲方业务部门单独开需求评审会，把需求一条一条过一遍。如果有业务逻辑或者字段不清楚的地方，我们会先记录下来，整理成问题清单，再发给甲方确认。有时候问题比较多，还会再约一次会议统一确认，最后形成正式的需求文档。

需求确定以后，我们还会跟甲方 IT 部门沟通技术方案。比如数据从哪些系统来，是 Oracle 还是 MySQL，有哪些接口能提供数据，现有平台支不支持改造，需要新增哪些表或者字段，最后整理成技术方案说明。

这两个文档都确定以后，我这边就开始编写 Mapping 文档。主要就是确定每一个字段来自哪张表、怎么转换、怎么关联，还有各层之间的数据流向和指标口径。

开发的时候，我们先把业务系统的数据同步到 Hive 的 ODS 层。我们项目里主要通过 Sqoop

把 Oracle、MySQL 的数据抽过来，DolphinScheduler 负责调度这些同步任务。

到了 DWD 层，主要就是做数据清洗和标准化处理，比如去重、过滤无效数据、处理空值，还有调用 UDF 对身份证号、手机号这些敏感信息做脱敏加密。

再往上到 DWS 层，就是按照客户主题、产品主题这些业务主题，把事实表和维度表关联起来，生成宽表。如果业务需要的话，也会做一些轻度汇总，比如团队累计保费、累计佣金、有效拜访人数这些，为后面的指标计算做准备。

最后到了 ADS 层，就是根据业务考核指标，把数据加工成报表需要的结果集，提供给 FineReport 或者 Quick BI 做数据展示。

整个开发完成以后，我们会提交测试，同步修改的问题，最后上线运行，后面也会持续关注任务调度和数据质量。”

主题：主要客户主题和产品主题

客户主题分为 内勤和外勤 收展员就在

项目具体指标

项目整体流程是这样的：第一阶段是需求调研与指标口径对齐，我们对接了销售管理部、人力资源部、保费部等部门，逐一确认老收展员月度达星、新人达 Q、有效拜访数、续保触达

率等考核指标的计算口径与统计维度，输出了需求规格说明书和四层 Mapping 文档，同时完成了数仓分层架构设计，统一了收展员工号、险种代码、机构编码等主数据标准。第二阶段是数据接入与 ETL 开发，我使用 DataX 从承保、佣金、CRM、考勤等系统抽取数据到 Hive ODS 层，在 DWD 层完成敏感字段脱敏和收展员业绩明细宽表构建，在 DWS 层完成核心考核指标加工，包括按月汇总 FYC 判定达星并计算连续未达星月数、按日累计新人件数与 FYC 判定达 Q 并预测月末达成概率、以及多粒度聚合有效拜访数和续保触达率等过程指标，最后在 ADS 层输出全员考核结果表，对接 FineReport 与 HR 薪酬系统。第三阶段是调度配置与测试上线，我利用海豚配置全链路 ETL 每日凌晨自动调度。

项目难点这块，主要有两个。第一个难点是考核指标口径统一复杂，销售管理部、人力资源部、保费部对达星等指标的理解不一致，且新老收展员考核规则差异大，我们采用自上而下定框架、自下而上逐项对齐的方法，最终输出经各部门签字确认的 Mapping 文档并存入 SVN 管理。第二个难点是数据质量保障难度高，源端存在员工号不一致、GPS 缺失、退保标记滞后等问题，CRM 与手工台账也常对不上，我在 DWD 层设计了工号补零对齐、佣金非负校验、GPS 空值填充、重复保单去重等清洗逻辑，并建立每日稽核脚本校验 FYC 总额一致性和关键字段空值率，发现问题自动告警。

数据来源和业务口径

维度与事实表

 维度表 (Dimension Table)

表名	说明	关键字段
dim_customer	客户维度	customer_id, name, id_card, phone, birth_date, gender, address
dim_product	产品维度	product_code, product_name, product_type, coverage_period, premium_rate
dim_agent	代理人维度	agent_id, name, hire_date, rank, team_id, status
dim_team	团队/机构维度	team_id, team_name, team_level, parent_team_id, manager_id
dim_date	日期维度 (通用)	date_id, year, quarter, month, week, day, is_holiday
dim_commission_rule	佣金规则维度	rule_id, product_code, rank, commission_rate, valid_start, valid_end
dim_activity_type	活动类型维度	activity_type_code, type_name, is_valid_visit, default_score

存在码表的话：比如银行汇率，产品实际名字。

 事实表 (Fact Table)

表名	说明	关键度量字段	关联维度
fact_policy	保单事实表 (核心)	premium (保费), sum_insured (保额)	customer, product, agent, date
fact_policy_transaction	保单交易事实表	transaction_amount (交易金额)	policy, date, transaction_type
fact_claim	理赔事实表	claim_amount (理赔金额)	policy, customer, date, claim_status
fact_payment	收付费事实表	payment_amount (收付金额)	policy, customer, date, payment_type
fact_commission	佣金事实表 (核心)	commission_amount (佣金金额)	agent, policy, date, commission_rule
fact_agent_activity	代理人活动量事实表	visit_count (拜访次数)	agent, customer, date, activity_type
fact_attendance	考勤事实表	attendance_count (参会次数), late_minutes (迟到分钟数)	agent, date
fact_kpi_score	KPI评分事实表	kpi_score (评分), ranking (排名)	agent, date, kpi_type

给 DeepSeek 发送消息

具体业务指标

达星

公司基本法规定，入司满 12 个月的老收展员，每个月要达到一定标准的 FYC，才能维持当前职级，这叫“达星”。如果连续不达星，就要降级甚至清退。

来源系统 人力系统/CRM (人员系统表)、佣金系统 (佣金明细表)、基本法配置表

数据口径

- 人员范围：入司日期距当前月份首日超过 12 个月的在职收展员
- 判断标准：当月 FYC $\geq$ 对应职级达星线(基本法配置表) (比如初级收展员是 3000 元，高级是 5000 元)

从贴源层获取 crm 和佣金表中每个业务员的当月总佣，过滤掉小于 12 个月的新人收展员，关联基本法配置表中的相应职级的达星线，caeswhen 若大于月佣金达星标准则达星，再通过 lag 开窗计算是否连续达星。

额外：连续问题

求连续两个月是否达星，lag 得到上个月的达星状态，判断本月是否达星且上个月是否达星。

Lag 向下偏移

方法一:lag 时间字段 偏移 1 lag_nextday 然后 lag_nextday - now_day 获取 daydif 最后 groupby 如果存在连续登录 n 天以上 count (difday) $\geq$ n 则连续；

方法二：lag 时间字段 偏移 n n 为 n 天 记 dif_day 判断 dif_day 是否等于 n 若相同则连续

连续登录问题

第一步：去重，对原始数据进行处理，对用户和登录日期（截取到天）进行去重，保证每个用户每天只有一条登录记录。第二步：开窗计算差值；在去重后的结果上，使用 OVER(PARTITION BY 用户 ORDER BY 登录日期) 进行开窗。再进行开窗 OVER() 取一个 RANK() 或 ROW_NUMBER() 值（因为第一步已经去重，所以函数结果相同）。核心操作：用 登录日期 减去 RANK 值的数值部分（得到一个“差值字段”）。连续登录的日期，其“差值”是相同的。第三步：分组筛选；将第二步的结果作为子查询，在外层使用 GROUP BY 用户，差值。使用 HAVING COUNT(*) >= N 来筛选出连续登录天数大于等于 N 的用户。例如，COUNT >= 3 表示连续登录 3 天以上。

达 Q

新人入司后有 6 个月保护期，保护期内累计完成一定的举绩件数和 FYC，就可以转正，这个叫“达 Q”。如果保护期满还没达 Q，就要淘汰。

来源系统 人力系统/CRM（人员系统）、核心业务系统（承保表）、核心业务系统（保单保费表）、基本法配置表（）

数据口径

- 人员范围：入司日期距当前日期不超过 6 个月的新人
- 判断标准：保护期内累计举绩件数 ≥ 达 Q 件数门槛（2 件），且累计 FYC ≥ 达 Q 保费 fyc 门槛（比如 1600 元）

从收展人员信息表中筛选出月份小于等于六个月的收展员信息 计算 int_date 小于等于 $int_date+180$ 在核心系统保费表当中收集每个业务员当月总 fyc 和合计保单数 根据基本法配置表判断每个业务员是否达到相应件数 (dq_standeran) 和相应 fyc (fyc_standanr)

有效拜访人数

收展员对客户进行的、符合公司认定标准的一次有效接触 , 通常要求停留时长和产生业务动作。

来源系统 CRM 系统 (拜访记录) 、 人力系统 (人员系统表)

数据口径

- 有效标准：在客户处扫码评价对应业务员，且见面及结束大于十五分钟，一日当中只统计首次扫码同一个客户为有效拜访
- 统计维度：日、周、月，按收展员+机构维度汇总

在拜访记录表里获取对应客户的 id 收展员相应 id 和时间等信息 过滤结束时间小于签到开始时间加十五分钟 (九百秒) 的数据 根据客户 id 去重统计日该收展员有效拜访个数

参会转化率

公司会举办产说会 (产品说明会) , 收展员邀请客户参加。邀请来的客户中，非重复的到场人数叫有效参会人数；参会的客户中最终成交的比例叫参会转化率。

来源系统 人力系统 (人员表)、活动管理系统 (会议信息表)、核心业务系统 (成交保单表)

数据口径

在会议信息表、客户邀请信息表和保单成交表中 获取到每个客户临会状态 再关联成交保单中是否有该客户 统计每个活动的客户成交率 参会后三十天内成交/有效参会人数

计算方式：过滤出三十天出席当天活动且三十天内出单 day in 三十天内的客户，count 当天参会且出单的客户，除有效参会人数.

有效参会率

描述收展员每日拜访人数，以确定收展员是否正常展业的指标，统计未达每周拜访数的收展员进行劝导和疑问处理。

数据口径

表在 考勤系统表 人员信息表

计算方式

计算公式 $\text{ROUND}(\text{实际参会天数} / (\text{应参会天数} - \text{免参会天数}) * 100, 2)$ 统计月参会率新人要大于百分之八十，老人百分之六十。

关键字段详细设计

1. 达星 (is_daxing) 与达 Q (is_daq)

这两个指标本质上都是月度业绩达标判定，区别在于阈值不同。

达星通常代表更高层级的业绩荣誉（如月度 FYC ≥ 3000 元），在部分保险公司还涉及星级人力津贴、养老基金计提等利益。

达 Q 通常是新人转正或维持最低考核标准的门槛（如月度 FYC ≥ 1800 元）。

FYC（First Year Commission，首年佣金）是保险行业的标准概念，来自承保系统的保单佣金字段。

统计周期：达星和达 Q 均为月度考核指标，在 Mapping 中需明确按自然月汇总。

2. 有效拜访（valid_visit_cnt）

这是典型的活动量管理指标，核心难点在于“有效”的定义。

有效拜访的通用定义：面对面拜访且能将销售流程推进到计划书制作/递送、促成、签单或有效资料收集等环节。

来源表：通常来自 CRM 系统或专门的活动量管理系统，记录每次拜访的类型、客户、结果等。

统计周期：通常为月度或周度，在 Mapping 中需根据业务需求明确统计周期。

3. 有效参会率（valid_attend_rate）

这是典型的日常行为管理指标，核心难点在于“参会”和“有效”的定义。

参会率通用公式：实到次数 $\div$ (应参会天数 - 免参会天数) $\times 100\%$ 。

应参会天数：通常为法定工作日，具体由公司制度规定。

有效参会：指参加职场早会及公司/部门正式会议，需排除培训、团建等活动。

常见考核阈值：参会率 $<80\%$ 则新人津贴系数为 0；参会率 $<60\%$ 则品管得分一票否决。

注意事项

业务口径确认是前提：达星/达 Q 的 FYC 阈值、有效拜访的判定标准、参会率的应参会天数定义等，都必须与业务方（甲方）正式确认并写入 Mapping 文档的“业务口径说明”列。

注意数据去重：有效拜访统计时，需用 COUNT(DISTINCT visit_id)避免重复计数；达星/达 Q 按月汇总时，需按 agent_id + 月份分组汇总。

注意时间边界：达星/达 Q 为月度指标，需明确统计周期（自然月）；参会率通常为月度，有效拜访可能是周度或月度，需在口径中明确。

NULL 值处理：无拜访记录或无保单的代理人，对应指标应设为 0 或 NULL，并在口径中说明（例如“无拜访记录则有效拜访次数=0”）。

工作中遇到的问题

下面这版是我按照真实面试回答的方式整理的，语气、逻辑都更像一个 3 年经验的数据仓库开发工程师在讲自己的项目，而不是背答案。

客户信息关联不上（编码问题）

客户信息关联不上（编码问题）

这个是在华安财险收展人员经营考核项目里面遇到的。

当时我们需要把 CRM 客户系统和承保系统的数据关联起来，生成客户经营分析的数据。其中客户姓名、身份证号这些字段都会作为关联条件。

后来开发的时候发现，有一部分客户始终关联不上，一开始我们还以为是 SQL 关联条件写错了，就把关联条件一条一条排查。后来抽取源数据进行对比，发现身份证号能够正常关联，但是客户姓名有部分出现了乱码。

接着我们继续排查数据同步环节，最后发现是两个业务系统使用的字符编码不一致，一个使用 UTF-8，一个使用 GBK，在数据同步读取的时候导致部分中文字段已经发生乱码，所以后面关联自然就失败了。

后来我们调整了数据同步配置，统一字符编码，同时在数据清洗阶段增加字符集转换，后续也在 Mapping 文档中明确了各字段的编码规范，之后新增的数据源也统一按照这个标准执行，后面就没有再出现类似的问题了。

客户重复、保费统计异常

客户重复、保费统计异常（数据标准化问题）

还有一个问题也是在收展人员经营考核项目里面遇到的。

当时我们需要统计收展人员服务客户数、累计保费、续保客户这些经营指标。

后来业务验收的时候发现，客户数量和保费金额跟业务部门统计的数据存在差异。

我们先把报表数据和源系统进行对比，最后发现问题主要出现在客户手机号和保费字段上。

不同业务系统保存手机号的格式不一样，有的是 11 位手机号，有的带+86 前缀，还有的带横杠，甚至还有 NULL、空字符串这些情况，所以同一个客户没有识别成同一个人。

另外保费金额也有问题，有些业务系统存的是"元"，有些存的是"分"，导致部分报表金额直接放大了一百倍。

后来我们就在 DWD 层统一做数据标准化处理，手机号统一去掉特殊字符和+86 前缀，只保留 11 位手机号，不符合规则的数据统一标记异常；空字符串和字符串 NULL 统一转换成真正的 NULL。

保费这块，我们和业务确认口径以后，统一把金额转换成"元"，并在字段说明中明确金额单位。之后客户数量和保费指标就和业务保持一致了。

Hive 任务执行很慢（数据倾斜）

Hive 任务执行很慢（数据倾斜）

还有一次是在开发收展人员经营分析宽表的时候。

当时需要把保单事实表和收展人员维度表做关联，正常情况下几十分钟就能跑完，但是有一次任务一直卡在 99%，两个多小时都没有结束。

一开始我们怀疑是不是集群资源不足，后来查看 Hive 执行日志和任务运行情况，发现其实是数据倾斜导致的。

当时有一个渠道下面的收展人员数据量特别大，占了整个数据量的大部分，所以所有数据都集中到了同一个 Reduce 任务，其他 Task 很快执行完成，但是这个 Task 一直处理不完。

后来根据业务情况进行了优化。

如果关联的是收展人员维度这种小表，我们就开启 MapJoin，让小表直接加载到 Map 端执

行，避免 Shuffle 过程。

如果确实是大表关联大表，我们会针对热点 Key 做单独处理，或者采用加盐的方式把热点数据打散，最后再进行合并。

优化完成以后，这个任务执行时间从原来的两个小时左右缩短到了十几分钟，后续运行也一直比较稳定。

额外可能会问

除了技术问题，其实项目里面更多遇到的是业务口径问题。

有一次做收展人员经营考核报表的时候，业务部门反馈我们的有效客户数比他们少了几十个。

一开始大家都觉得是 SQL 写错了，我们把数据一条一条核对以后才发现，其实是双方对"有效客户"的定义不一样。

我们最开始统计的是保单生效客户，而业务部门统计的是保单生效并且没有退保的客户，所以最终结果存在差异。

后来我们重新和业务确认指标口径，在 DWS 层增加了有效客户的统一判断逻辑，并把口径同步更新到 Mapping 文档，后面所有报表都按照统一标准计算，之后就没有再出现类似的问题了。

额外指标（应对面试可能会问）

- 客户旅程：覆盖投保、理赔、保全等关键旅程

- 代理人留存率
- 个险新单期交保费
- 团队总保费

第二个项目

对客户集中统一分析 从不同的系统当中对其业务口径 对客户实行精准营销

项目整体流程是这样的 :第一阶段是需求调研与规划 ,我们对接了市场部、续期部、理赔部、客服部 ,逐一确认客户价值分、续保意向评分、产品偏好、理赔风险、流失预警等标签的计算口径与更新频率 ,输出了需求规格说明书 ,同时完成了 MaxCompute 数仓分层设计和 ODS 到 ADS 各层表结构设计 ,统一了以客户 ID 为核心的打通规则。第二阶段是数据接入与 ETL 开发 ,我使用 DataX 从承保、理赔、续期、客服工单、官网埋点 5 套系统抽取数据到 MaxCompute ODS 层 ,在 DWD 层完成清洗脱敏和客户全景宽表构建 ,在 DWS 层完成 RFM 价值分层、续保意向、产品偏好、理赔风险、流失预警等标签计算 ,最后在 ADS 层输出客户标签宽表和分群推荐表 ,并用 Quick BI 开发了画像看板、分群地图、流失预警图等 10 余张报表。第三阶段是调度配置与测试上线 ,我利用 DataWorks 配置全链路 ETL 任务调度 ,保障每日 T+1 数据刷新 ,最后独立完成自测并协助 SIT 集成测试。

项目难点这块 ,主要有两个。第一个难点是标签口径统一复杂 ,市场、续期、理赔等部门对客户标签的定义和划分标准不一致 ,跨部门对齐成本高 ,我们采用自上而下定框架、自下而上按数据可算性填充的方法 ,形成可配置的标签模型。第二个难点是数据质量保障难度高 ,源端存在客户信息缺失、历史数据不一致、日志无效流量等问题 ,影响画像准确性 ,我在 DWD 层设计了校验脱敏、去重过滤等清洗逻辑 ,并建立了标签覆盖率、ID 打通率等质量稽核规则 ,确保标签准确率不低于 95%。

第三个项目

与需求调研，与信贷、风控、人力等部门沟通，将业务目标转化为累计贷款发放额、不良贷款率、逾期率等核心考核指标，编写 ODS 到 ADS 四层 Mapping 文档

项目整体流程是这样的：第一阶段是需求调研与指标口径对齐，我们对接了信贷管理部、风险管理部、人力资源部等部门，逐一确认累计贷款发放额、不良贷款率、逾期率、拨备覆盖率等核心考核指标的计算口径与统计维度，输出了需求规格说明书和 ODS 到 ADS 四层 Mapping 文档，同时统一了联社编码、贷款五级分类、客户经理工号等主数据标准。第二阶段是数据接入与 ETL 开发，我使用 DataX 将源端 OceanBase 中的核心业务与信贷管理系统数据同步至 Hadoop HDFS，使用 Sqoop 抽取 MySQL 系统中的辅助数据至 Hive ODS 层，在 DWD 层构建覆盖放款、还款、分类、担保的贷款全生命周期明细宽表，在 DWS 层完成累计放款额、贷款余额、不良率、逾期率、拨备覆盖率及客户经理绩效分等指标汇总，最后在 ADS 层输出业绩考核表与资产质量监控表，并用 FineReport 连接 Hive 开发了信贷投放进度、不良率趋势、联社资产质量排名、监管报送指标等看板，SQL 脚本与报表模板通过 SVN 管理。第三阶段是调度配置与测试上线，我利用 Dolphin Scheduler 编排全链路 DAG 工作流，实现每日自动调度与告警，并编写 Shell 脚本配合 Hive SQL 进行贷款总额一致性与不良率口径合规性稽核，每日跑批后输出质量报告，最后独立完成自测。

项目难点这块，主要有两个。第一个难点是信贷指标口径统一复杂，信贷、风控、人力等部门对不良率、逾期率等指标的理解存在差异，比如不良贷款是否纳入展期或借新还旧情形，五级分类标准是否参照最新监管要求等，跨部门对齐成本高。我们的应对策略是采用自上而下定框架、自下而上逐项确认的方法，最终输出各部门签字确认的 Mapping 文档。第二个

难点是数据质量保障难度高，源端 OceanBase 和 MySQL 系统中存在贷款状态更新滞后、五级分类缺失、历史数据迁移不完整等问题，直接影响不良率和拨备覆盖率的准确性。我的应对策略是在 DWD 层设计贷款状态校验、五级分类补全、重复记录去重等清洗逻辑，并通过每日稽核脚本校验关键指标的一致性，确保监管报送数据的准确率。

面试可能问题

地点

大学情况：大学大四离校 在 23 年 3 月份去 中途项目赶工期投产 线下考试没去 12 月份

拿毕业证

公司地址情况：





项目人员配置情况：

小组有 8 个人（驻点）一个小组长带队、一个全栈负责 java、一个测试（联系甲方）、五个数据开发。

公司项目开发形式:一个公司二十五个左右 我所在的项目小组有五个人 一个经理和经理助理 助理负责分发工作到各个小组 公司主营业务一般在金融领域 主要是本省的 保险、银行、制造业也有一部分。

延毕问题

当时在实习 项目正好赶上要投产 所以没空去考那门试 导致延毕

去贵州的原因

广东就业难度较大 封校期间出去实习机会比较少 刚好当时赶上解封 想着往外面去找找工作 所以跑去贵州

汇总层逻辑：核心逻辑包括：老收展员达星判定（按月汇总入司超 12 月人员的 FYC，自动

标记是否达星并统计连续未达星月数)、新人达 Q 追踪(按日累计新人举绩件数与 FYC,判定转正条件)、以及有效拜访数、等过程指标的多周期汇总

总结

一、最细颗粒度的表(事实表)

最细颗粒度指的是**每行记录代表最底层、最原子化的事件或明细数据**。从项目描述来看,以下几张表属于最细颗粒度:

1. 活动记录表

每行代表一次具体的客户拜访或活动记录。颗粒度为**单次活动**,记录收展员每次拜访客户的时间、方式、内容等明细信息。这是支撑“有效拜访客户数”指标的最原始数据,粒度最细,数据量最大。

2. 保单表

每行代表一张保单的明细信息。颗粒度为**单张保单**,记录每笔保单的投保人、险种、保费、签单日期等明细信息。这是业绩考核和佣金计算的核心数据来源,也是后续保费规模分析的底层明细表。

3. 佣金表

每行代表一笔佣金发放的明细记录。颗粒度为**单笔佣金**,记录每笔佣金的发放对象、金额、对应保单、发放日期等明细信息。这是考核收展员收入及业绩产出的最细颗粒度数据,直接支撑佣金相关的考核指标。

4. 代理人信息表

每行代表一个代理人(收展员)的基本信息。颗粒度为**单个代理人**,记录收展员的工号、姓

名、所属团队、入职日期、职级等信息。虽然它承载的是属性信息，但在数仓建模中属于**维度表**，用于描述事实数据的人员属性。判断依据是：该表主要提供“谁”的属性信息，被多张事实表关联使用。

二、维度表

从项目描述中识别出以下维度表：

1. 代理人信息表

代理人（收展员）的基础属性，包括工号、姓名、所属团队、职级、入职日期等。用于分析各团队、各职级人员的考核情况。

2. 客户经营信息表

客户的基础属性信息，包括客户号、姓名、联系方式、地区等。用于关联保单表和活动记录表，分析客户维度的经营情况。

3. 团队/机构维度表

收展员所属团队或机构信息，包括团队编号、团队名称、上级团队等（描述中隐含“代理人团队”字段）。用于按团队维度进行业绩汇总和排名。

4. 时间维度表

日期、月份、季度、年度等时间属性信息，包括日期、年、季、月、周、是否工作日等（描述中明确提到“每日、每周、月度、季度、年度”统计维度）。用于按不同时间粒度对考核指标进行汇总分析。

5. 产品维度表

保险产品的分类属性信息，包括产品代码、产品名称、险种大类等（从保单表中可关联获取）。用于分析不同产品的销售情况。

三、判断依据

区分事实表与维度表的核心标准是“谁在描述谁”：

判断维度	事实表（明细表）	维度表
内容	记录事件、行为、业务过程，描述“发生了什么”	记录属性、分类、字典，描述“谁、什么、哪里、何时”
颗粒度	通常较细，代表最原子化的事件记录	通常较粗，代表一个维度实体
更新方式	通常增量追加	通常拉链/全量更新
数据量	通常较大	通常较小
典型字段	数值型指标（金额、数量）+ 外键	描述型字段（名称、分类、层级）

四、数仓模型结构

结合上面的分析，该项目的数仓模型可梳理为以下结构：

DWD 层：

- 事实表：保单表（单张保单粒度）、佣金表（单笔佣金粒度）、活动记录表（单次活动粒度）
- 维度表：代理人信息表、客户经营信息表、团队维度表、时间维度表、产品维度表

DWS 层：

- 基于 DWD 层按“人（收展员）× 日”粒度汇总，计算有效拜访客户数、参会次数、保费规模等指标
- 按“人（收展员）× 月”粒度计算达星、达 Q 等月度考核指标

- 按“团队 × 月”粒度汇总团队整体业绩

ADS 层：

- 考核结果明细表
- 多维度可视化看板

这种模型设计遵循了“维度建模”的经典思路：将业务过程记录为事实，将业务实体和分类抽象为维度，通过外键关联实现灵活的多维分析。

优化：

针对大宽表多表连接导致的性能瓶颈，采用按“代理人团队”字段进行分桶处理，将大任务拆分为多个并行子任务，减少全表扫描与 Shuffle 开销，显著提升 Hive 查询效率。

五、老师小结

1.hive 的架构和工作原理。

hive 架构由 hdfs mr 和 yarn 构成 具体工作原理在 mr 里首先通过解析器分析语句

从 Metastore 里面获取查询到相应的表格和元数据 再将语句和获取的元数据给编译器

编译器会将查询语句的所有可能的实现语句做出来 再分配给优化器

优化器会选择消耗硬件资源最少的方式分配给执行器执行查询语句

2.什么是 SMB join ?

将 a b 表的 sid 进行分桶且开启 roc 分桶后排序 分桶后 ab 表的数据是一一对应的

这样操作可以减少在合并表的时候极大减少对于硬件资源的占用 并且会极大加快 join 速度

3.Hive 调优的方向有哪些 sql 调优

1 分桶表可启用 SMB Join

2 采用 ORC/Parquet 列式存储

3 先过滤后关联

4 分区裁剪、列剪枝、提前过滤

本质：减少数据扫描量、避免 Shuffle 倾斜、利用分布式并行能力

4.数据倾斜

原本应该均匀分配给多个机器并行处理的数据，因为某个点数据量过大，

导致大部分任务 1 分钟跑完，而处理这个 Key 的任务跑了 1 小时甚至 OOM (内存溢出) 崩溃。

解决：

1. 打散空值 CASE

```
WHEN user_id IS NULL THEN CONCAT('null_', RAND())
```

```
ELSE user_id
```

2. 开启 Hive 的倾斜优化参数 -- 开启 Map 端聚合(会在 Map 端先做一次部分聚合，

减少 Reduce 数据量)

```
set hive.map.aggr = true;
```

-- 开启 Skew Join (自动检测倾斜键 , 将其拆分成多个任务处理)

```
set hive.optimize.skewjoin = true;
```

3. 使用分桶当大表 join 大表时 ,减少数据倾斜出现 ;当大表 join 小表且小表小于 25mb 时 启用 mapjoin 在本地进行 reduce 操作 减少对 shuffle 的占用 (“在 Map 端完成 Join , 不走 Reduce”), 但是如果数据过大会造成内存溢出 (oom) 这时候只能使用加盐或者分桶去避免数据倾斜

Hive 的数据倾斜 :

1 GROUP BY 引发的数据倾斜 : 可以开启 Map 端聚合配置负载均衡 : hive 生成的查询计划会有两个 mapreduce job 来执行 group by 操作 , 第一个 map 的输出结果按照 key 随机分配到不同的 reduce 中 , 每个 reduce 只能对同一个 key 做部分聚合操作 , 从而避免数据倾斜 , 达到负载均衡的目的 (随机)。第二个 map 的输出结果会保证相同的 key 分配到同一个 reduce , 最终完成 key 的聚合操作 (统一)。

2 count (distinct) 引发的数据倾斜 : 使用 group by+count 语句代替 ; 第一层 select 通过 group by 语句对字段去重操作 , 第二层 select 再通过 count 语句统计个数 , select count(字段) from (select 字段 from table group by 字段) t;

3 空值引发数据倾斜 : 避免空值参与关联 (where 过滤掉); 随机赋值。

4 不同数据类型关联引发的数据倾斜 : 通过 cast 操作将字段转换成相同的数据类型 ;

5 不可拆分的大文件引发的数据倾斜 : 在数据压缩的时候采用 bzip2 和 zip 等支持文件分割的压缩算法。

6 表关联引发的数据倾斜：进行 mapjoin。

7 map 和 reduce 的设置不合理：重新设置 map 和 reduce。

8 使用万能开关解决异常数据问题。

5.开窗函数

1.什么场景需要用开窗函数

partition by 与 group by 的区别

partition by 是属于多进多出 group by 多进一出

partition 主要用在需要处理排序 输出指定排序上

group by 可以用于字段去重和配合聚合函数的使用上

2.三大部分

-行控制的聚合

行聚合控制 sum avg count max min

不开窗下就是多进一出

LAG,LEAD

偏移函数 向上偏移和向下偏移

-排序问题

开窗函数排序三个

rank 重复且连续

desnkrank 重复不连续

row number 连续不重复

常用星型维度表将与主维表关联的若干张相关维表关联到一起作为一张宽表

确定维度属性 在一张维度表里尽可能丰富维度表的字段 避免后续每次处理

7.什么叫数据探查

数据探查是数据汇聚阶段的首要环节，目的是在正式接入数据前，描述数据源的数据质量情况

对源数据的内容、结构、质量进行预分析，为后续的数据定义、挂接、读取提供依据。

通过人工方式或智能化技术，对源数据进行内容分析、结构分析、质量评估，输出探查报告，指导数据治理工作。

8.常见的数据的清洗数据脱敏规则有哪些

对于敏感字段比如名字 手机号 身份证号等

在字段里面通过自定义函数转换变成带星号之类的字符串形式

清洗规则：

1、空值处理：

空值替换：银行客户表里的职业为空，填'未知'；交易金额为空，填-999 或 0

强制非空：如果关键业务字段（如订单号、账户 ID）为空，直接丢弃该行

2、格式规范化：

去除前后空格：TRIM() 去掉字符串首尾的空白字符

统一大小写：英文统一转为 UPPER() 或 LOWER()

日期格式化：将 2023/01/01、01-01-2023 统一转为 yyyy-MM-dd

衍生与映射：

3、字典映射：将源系统的业务代码，映射为数据仓库的标准代码

常见的清洗：

1、对 XM(姓名)、SFZH(身份证号)、LXDH(联系电话)、DZ(地址)、SJ(时间)等字段做清洗操作。

2、对测试数据进行清洗。

3、对业务重复数据进行清洗。

4、对数据做格式转换。

5、业务代码和标准代码转换。

9.拉链表

1 适用范围 缓慢变化维

2 比源数据多两个字段 开始生效时间和结束生效时间

3 位置 公共维度表

4 分区 以结束生效时间字段作为分区

5 与之对应贴源层为增量更新

10.拉链表更新表流程

拉链表的话，我们项目主要是用来保存维度数据的历史变化，比如客户信息、收展人员归属这些。

更新的时候，假设今天是 T 日，我们处理的就是昨天产生的增量数据。首先会把昨天新增和修改的数据同步到 ODS 层，然后拿 ODS 的数据去和 DIM 层当前生效的数据做关联比对。

如果数据没有变化，就继续保留；如果数据发生了修改，就把原来那条记录的结束日期改成昨天，再新增一条最新的数据，开始日期也是昨天，结束日期还是 9999-12-31，表示当前生效；如果是新增数据，就直接插入一条新的生效记录。

最后通过 INSERT OVERWRITE 更新拉链表，把历史数据和最新数据一起写回。

我理解拉链表最大的作用，就是既能查询当前最新的数据，又能保留历史变化记录。像保险项目里面客户归属、客户等级这些信息都会变化，所以比较适合用拉链表来管理。

数仓建模 = mapping 与宽表建模 + 维度模型 (事务表+维度表)

数仓建模流程逻辑

1 确定目标表结构信息

从 BI 确定需要的目标表结构事务和维度字段

2 调研梳理来源表信息

在确定好事务和维度字段之后从后端数据库整理事务和各个维度相应的表

3 调研确认应用业务口径

调研事务需要使用什么方法去输出需要显示的方式

4 基于模型做取值映射分析

上面三步完成之后对后端数据库获取的数据在 dm 层进行映射到相应需要输出的字段

Mapping

三个共同：业务过程相同、周期相同、统计粒度相同

本质：描述数据从来源到目标的“流转地图”，记录字段级血缘关系

1 目标字段 DM、DWS 层表结构

2 来源字段 ODS、DWD 层字段

3 转换规则 空值处理 NVL、类型转换 CAST、计算逻辑 SUM/CASE WHEN

4 业务口径说明 如“佣金=保费*提点

宽表建模

本质：将多个维度表的属性冗余到事实表中，形成“大宽表”，避免查询时多次 JOIN

1 选择公共粒度：如按 customer_id (客户) + salesman_id (销售员) + ds 组合，保证每行唯一。

2 冗余常用维度属性：受益人、被投保人、产品名称、地区名称等直接存入宽表

3 预计算轻度聚合：如 sum(fyc_amount)、count(distinct customer_id)，为 DM 层加速

统计本月成交用户：

有多少客户

有多少投保客户

有多少活跃客户

有多少收展服务客户

有多少成交客户

优点：查询性能好（减少 JOIN），易理解（一张表涵盖所有分析维度）

维度模型

一

事务型事实表 四步

选择业务过程 选择什么业务比如退款表作为业务过程 用户申请退款 商家审核 系统退款

成功 这个过程作为业务过程

声明颗粒度 以某个字段为最细字段 比如一个人一次性买了两件商品 可以退部分也可以退

全部 所以可以将每件商品项的退款作为最细颗粒

确认维度 比如确定具体退款时间、退款审核人、退款方式、退款原因等维度

确认事实 确认事实就是哪些字段可以聚合的字段 比如退款总金额 退款总商品数等

二

维度表设计

确定维度 由于可能存在多个事实表与统一维度表相关 为了保证维度唯一性 只创建一张维

度表

确定主维表和相关维表的关系 最细的表作为主维表 主维表可以直接关联事实表 通

14 7-9 个银行系统名称

cbs 核心系统 (核心业务)、ecif (客户管理系统)

ccp (信用卡系统)、bbs (票据系统)、ccs (对公信贷管理系统)

pcs (零售信贷系统)、nbs (网上银行系统)

15 数据治理 (管理数据管理)

1 元数据管理 管理数据管理的工作，实现标准化 (按照规则)，模板话 (输出文档)，自动化 (利用工具)

(1) 元数据管理：数据字典和 Mapping 是元数据管理中最核心的两个落地产物

1) 数据字典 (静态)：数据结构 (字段是什么)

2) mapping: (动态)：血缘追溯 (字段从哪来)

2 主数据管理 是支持业务过程发生的必要的实体数据 (业务过程的维度字段)

3 数据标准管理 制定数仓过程全流程的标准

4 数据质量管理 包含数据探查、数据清洗

5 数据安全管理 通过详细的权限管理进行安全管理

6 数据资产价值管理 整个数据管理的核心 (通过实现数据资产价值目录来实现资产价

值，通过描述哪些资产在哪里来检索资产)

负载均衡

是一种将网络流量或工作负载按照某种策略分发到多个服务器 (或服务实例) 上的技术，旨在优化资源利用率、最大化吞吐量、最小化响应时间，并避免单点过载。

你可以把它想象成大型超市的收银台：如果只开一个收银台，排队就会很长；如果根据排队人数动态开启多个收银台（并让排队最短的顾客去空闲的收银台），整体结账效率就会大幅提升——这就是负载均衡的直观体现。

负载均衡 = 流量调度员 + 故障巡检员。它既负责把海量请求按策略分发给后端服务器，也负责及时发现并隔离故障节点，让整个系统具备高并发、高可用、易扩展的能力。

索引

就是一种用于帮助数据库系统快速查找数据的“目录”或“查找表”，在过滤条件中根据 rowid 快速定位，在实际开发中，**最左前缀匹配**和**列区分度**会导致索引失效，避免因查询写法不当导致索引失效。在数仓项目中，我们通常用**分区裁剪**和**分桶**来替代传统索引，以适应海量数据的计算场景。

MySQL 索引失效的经典场景定义

1. 最左前缀原则失效

在联合索引中，查询条件必须从索引的最左列开始，且不能跳过中间的列。

2. 隐式类型转换失效

当索引列的字段类型与查询条件传入的数据类型不一致时，MySQL 会自动进行类型转换。

3. 索引列上使用函数或表达式计算失效

当 WHERE 条件中对索引列使用了函数运算（如 DATE()、YEAR() 等）或数学表达式计算（如 +、-、*、/ 等）时，索引列的值被改变，MySQL 无法基于原始值进行快速定位，索引失效。

4. LIKE 模糊查询以通配符（%）开头失效

B+Tree 索引按照从左到右的顺序排列，当 LIKE 模式以通配符 % 开头时，索引的左侧无法确定起点，导致无法使用索引进行范围查找，索引失效。

5 优化器判定全表扫描优于索引失效

当查询结果集占总数据量的比例过高（通常超过 20%~30%）时，由于走索引需要进行“回表”操作产生大量随机 I/O，而全表扫描采用顺序 I/O 成本更低，MySQL 优化器会主动放弃索引，选择全表扫描。此外，统计信息过时也会导致优化器误判，造成索引失效。

数据治理实现三化

标准化（制定统一的数据命名、字段类型、业务口径和编码规则。

如订单金额字段必须叫 `order_amount` ,单位统一为“元”。目标是让数据“长一个样”,消除歧义。)

模版化 (将标准化的流程 (建表、ETL 映射、清洗逻辑) 沉淀为可复用的模板文件。

如 `dim_sku_info_df` 建表语句固定格式,插入逻辑沿用统一模板。目标是“一次编写,多次复用”,提升效率。)

自动化 (基于模板和标准,通过调度工具 (如 Azkaban) 自动执行数据接入、清洗、汇总等任务,并配置质量校验和告警机制,

无需人工干预即可产出可信数据。目标是“降本增效、释放人力”。)

16 银行主题 四个主题 每个主题下面还有二三级主题详细 看连接

<https://www.cnblogs.com/zourui4271/p/13877676.html>

十大金融主题 当事人、产品、协议、事件、资产、财务、机构、地域、营销、渠道

存储文件形式

由于数据库过来的数据比较大,优化之一 贴源层使用 `parquet` 其他层使用 `textfile`

● Hive 有关内容

hive

解析器去 ms 获取元数据到编译器

编译器 (编译执行计划) 后给优化器浪费硬件资源最少地执行计划给执行器

执行器翻译语句给 mr 执行计算

建表当中 冒号和下划线、空格之类的都是元数据

struct 中 key 是元数据 values 是真实数据

外部表

先创建再删除再创建一模一样的表可以找到原来的真实数据

--如果非事务表之类的想插入或者修改数据

可以先查询出 hive 表中的数据或者先用查询语句修改数据集后再通过 overwrite

覆盖原表从而达到 ddl 的 update ddl 更新效果

分区分桶的区别

分区键不存在真实数据文件中 但是可以在 hive 查询到 可以作为一个真实字段 称为伪列

分区扫描 加快扫描 分区列相当于一个文件

分桶字段是真实存在于数据文件中

分桶 smb join 一个桶代表一系列文件

分区就是分目录加快查询速率 分桶就是分文件提高表关联速度

Azkaban

project 可视化 web 页面

flrow zip 压缩包

job job 脚本

依赖 dependencies=job

面试题：在 azkaban 当中如果跨流跨 job 执行完 step1.job 后需要判断外部 hello.job 是否完成执行

再执行 step2.job 在 liunx 脚本中如何实现判断外部 job 是否执行完成的代码

逻辑：在外部 job 的 shell 中输出一个文件 再在 setp2 的 shell 当中进行时间间隔

循环 若不存在则循环知道存在

若存在则删除文件再执行 step2 job

每个 job 就是一个执行文件 执行文件会执行封装好的 shell 脚本 从 shell 脚本调用相应代码 执行顺序是

先从 job 获取对应封装的脚本 再在脚本里面判断是否执行或者直接执行 shell 的脚本程序；

最细的是主维表 主卫表直接与事实,表关联

维度 dim 的模型

星型模型 雪花模型

主要是星型模型.

exists 与 in 子查询的区别

若子大主小 用 exists 子查询

若子小主大 用 in 子查询

hive 排序

Task 排序

order by 只适合小数据排序 不适合大数据排序

sort by 局部有序 全局无序 各自 reduces 内排序

distribute by 根据字段分组 然后用 sort by 排序

当分组数小于等于 reduces 数 则根据分组排序 当大于 reduces 数 则随机数随机分组排序

cluster by 是 distri by 与 sort by 结合 字段要相同才能用

● 退链操作

先删除生效时间大于问题时间的数据的所有失效分区

acct_num	info	start_date	end_date	is_current	变化说明
A001	状态1	2020-01-01	2020-09-30	N	不变
A001	状态2	2020-10-01	2020-10-07	N	不变
A001	状态3	2020-10-08	2020-10-08	N	不变
~~A001~~	~~状态4~~	~~2020-10-09~~	~~9999-12-31~~	~~Y~~	被删除
~~A002~~	~~新开户~~	~~2020-10-10~~	~~9999-12-31~~	~~Y~~	被删除

恢复闭链 将结束时间大于问题时间的 end_date 的行设置为 9999-12-31

acct_num	info	start_date	end_date	is_current	变化说明
A001	状态1	2020-01-01	2020-09-30	N	不变
A001	状态2	2020-10-01	2020-10-07	N	不变
A001	状态3	2020-10-08	9999-12-31	Y	已恢复为当前有效
~~A001~~	~~状态4~~	~~2020-10-09~~	~~9999-12-31~~	~~Y~~	已被删除（上一步）
~~A002~~	~~新开户~~	~~2020-10-10~~	~~9999-12-31~~	~~Y~~	已被删除（上一步）

总结：正确的退链条件

步骤	条件	说明
删除	<code>start_date > T</code>	删除 T 之后新增的记录
恢复	<code>end_date >= T</code>	恢复 T 当天及之后闭链的记录（因为 T 时刻的那条记录也需要回到当前状态）

Sql 调优

索引优化

避免索引失效 索引字段隐式转换, not in not exists 判断导致失效, 索引列参与计算, 使用函数, 索引列模糊查询,

sql 写法优化: 避免在 select 后面使用*, in not in 用 exists not exist 替换
少用 join, 用速度快的 unionall 替换 union, 用 group by 代替 distinct 去重
使用 sql 内置函数比如用 decode 替换 case when 加快查询速度
对某段 sql 用 with as 创建临时表 拆分步骤
排序字段建立联合索引, 不必要的排序直接删掉

如何查看执行计划

执行计划是一条查询语句在 oracle 中的执行过程或访问路径

执行完一条 select 语句后, 在 plsql 执行计划窗口查看, 或按 f5 查看

或者在命令窗口输入 sql>explain plan for select * from table;

```
Sql>select * from table(dbms_xplan.display());
```

主要查看 1 执行顺序: 缩进最多的先执行; 同一级遵守最上最右先执行。

2 表扫描: 全表、索引

3 关联机制: 嵌套、哈希、排序

4 耗费 (cost)、cpu 耗费: 理论上越小越好

存储过程异常有哪些？

IN 表示传入参数，不可以被赋值，OUT 表示传出参数，可以被赋值，IN OUT 表示传入传出参数，可以被赋值，可以返回值

76 常用的异常都有哪些？

异常名称	异常码	描述
DUP_VAL_ON_INDEX	ORA-00001	试图向唯一索引列插入重复值
INVALID_CURSOR	ORA-01001	试图进行非法游标操作。
INVALID_NUMBER	ORA-01722	试图将字符串转换为数字
NO_DATA_FOUND	ORA-01403	SELECT INTO 语句中没有返回任何记录。
TOO_MANY_ROWS	ORA-01422	SELECT INTO 语句中返回多于 1 条记录。
ZERO_DIVIDE	ORA-01476	试图用 0 作为除数。
CURSOR_ALREADY_OPEN	ORA-06511	试图打开一个已经打开的游标

77 如何找到存储过程哪个环节执行的快慢？

用日志是用来追溯问题的，记录整个程序的运行情况，知道哪个环节报错了，记录每一步花了

锁、死锁怎么处理

事务是一组数据库操作，要么全部成功，要么全部失败，具有 ACID 四个特性：原子性、一致性、隔离性和持久性。

锁是数据库为了保证并发访问数据的一致性而使用的机制，常见有共享锁（读锁）和排他锁（写锁）。共享锁（S 锁、读锁）：多个事务可以同时读取数据，但不能修改。

排他锁（X 锁、写锁）：一个事务修改数据时，其他事务不能读写该数据。

死锁是多个事务互相等待对方释放锁，导致事务无法继续执行。数据库一般会检测死锁，并回滚其中一个事务来解除死锁。开发过程中可以通过统一加锁顺序、缩短事务时间、及时提交事务以及建立合适索引等方式减少死锁的发生。